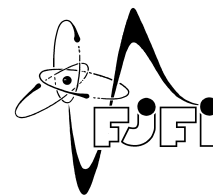




CZECH TECHNICAL UNIVERSITY IN PRAGUE
Faculty of Nuclear Sciences and Physical Engineering



Procedural generation of dungeons in Godot Engine

Procedurální generování dungeonů v Godot Engine

Bachelor's Degree Project

Author: **Robin Havlina**
Supervisor: **Ing. Petr Pauš, Ph.D.**
Language advisor: **Bc. Nathaniel Tobias Patton**
Academic year: 2025/2026

I. OSOBNÍ A STUDIJNÍ ÚDAJE

Příjmení: **Havlina** Jméno: **Robin** Osobní číslo: **519842**
Fakulta/ústav: **Fakulta jaderná a fyzikálně inženýrská**
Zadávající katedra/ústav: **Katedra matematiky**
Studijní program: **Aplikovaná informatika**

II. ÚDAJE K BAKALÁŘSKÉ PRÁCI

Název bakalářské práce:

Procedurální generování dungeonů v Godot Engine

Název bakalářské práce anglicky:

Procedural generation of dungeons in Godot Engine

Jméno a pracoviště vedoucí(ho) bakalářské práce:

Ing. Petr Pauš, Ph.D. katedra softwarového inženýrství FIT

Jméno a pracoviště druhé(ho) vedoucí(ho) nebo konzultanta(ky) bakalářské práce:

Datum zadání bakalářské práce: **31.10.2025**

Termín odevzdání bakalářské práce: **03.08.2026**

Platnost zadání bakalářské práce: **30.09.2027**

podpis vedoucí(ho) ústavu/katedry

podpis proděkana(ky) z pověření děkana(ky)

III. PŘEVZETÍ ZADÁNÍ

Student bere na vědomí, že je povinen vypracovat bakalářskou práci samostatně, bez cizí pomoci, s výjimkou poskytnutých konzultací. Seznam použité literatury, jiných pramenů a jmen konzultantů je třeba uvést v bakalářské práci.

_____ Datum převzetí zadání

Havlina Robin
_____ Podpis studenta

I. OSOBNÍ A STUDIJNÍ ÚDAJE

Příjmení: **Havlina** Jméno: **Robin** Osobní číslo: **519842**
Fakulta/ústav: **Fakulta jaderná a fyzikálně inženýrská**
Zadávající katedra/ústav: **Katedra matematiky**
Studijní program: **Aplikovaná informatika**

II. ÚDAJE K BAKALÁŘSKÉ PRÁCI

Název bakalářské práce:

Procedurální generování dungeonů v Godot Engine

Název bakalářské práce anglicky:

Procedural generation of dungeons in Godot Engine

Pokyny pro vypracování:

Cílem práce je navrhnout a implementovat systém procedurálního generování herních dungeonů v enginu Godot s využitím modulárního přístupu. Systém vygeneruje náhodný dungeon z předem vytvořených místností, které jsou propojeny podle pravidel, tak aby zaručila hratelnost a logiku hry. Funkci generátoru potvrdí prototyp jednoduché hry s pixelovým stylem.

Pokyny/Postup:

1. Provedení rešerše existujících metod generování herních úrovní se zaměřením na modulární generování.
2. Analýza typické struktury u dungeonů a návrh pravidel, které definují, jak propojit jednotlivé místnosti a jejich generování.
3. Vytvoření algoritmu generujícího náhodný dungeon z předem připravených místností v enginu Godot.
4. Vytvoření prototypu hry, která tento generátor bude používat.
5. Testování a vyhodnocení funkčnosti generátoru.

Seznam doporučené literatury:

1. Bradfield, C., Godot 4 Game Development Projects. Packt Publishing, 2023.
2. Vanhove, S., Learning GDScript by Developing a Game with Godot 4: A fun introduction to programming in GDScript 2.0 and game development using the Godot Engine. Packt Publishing, 2024.
3. Oficiální dokumentace Godot (Godot Docs). URL: <https://docs.godotengine.org>

DECLARATION

I, the undersigned

Student's surname, given name(s): Havlina Robin
Personal number: 519842
Programme name: Applied Informatics

hereby declare that the Bachelor's Thesis titled

Procedural generation of dungeons in Godot Engine

is my own independent work and that I have declared all sources of information used in accordance with the Methodological Guideline for adhering to ethical principles when elaborating an academic final thesis and the Framework Rules for the use of artificial intelligence at CTU for study and pedagogical purposes in Bachelor and continuing Masters studies.

I declare that I have used artificial intelligence tools during the preparation and writing of the final thesis.
I've verified the generated content. I confirm that I am aware that I am fully responsible for the content of the final thesis.

In Prague on 16.07.2026

Robin Havlina

.....
student's signature

Acknowledgment:

I would like to thank for (his/her expert guidance) and express my gratitude to for (his/her language assistance).

Procedurální generování dungeonů v Godot Engine

Studijní program: Aplikovaná informatika

Vedoucí práce: Ing. Petr Pauš, Ph.D., pracoviště školitele (název instituce, fakulty, katedry...)

Klíčová slova: klíčová slova (nebo výrazy) seřazená podle abecedy a oddělená čárkou

Procedural generation of dungeons in Godot Engine

[illegible]

Key words: keywords in alphabetical order separated by commas

Contents

Introduction	9
1 Theoretical Background of Procedural Content Generation	10
1.1 Foundations and Taxonomy of Procedural Content Generation	10
1.1.1 The PCG Taxonomy	10
1.1.2 Dimensions of Control and Parameter Vectors	11
1.1.3 The Challenge of Expressive Range	11
1.1.4 Advantages and Challenges of PCG	11
2 Technology Stack and State of Art	14
2.1 Godot Engine	14
2.1.1 The Node and Scene Composition	14
2.1.2 GDScript	14
2.2 Visuals and Pixel Art Aesthetic	15
2.2.1 The Role of Pixel Art in Roguelike Design	15
2.2.2 Aseprite	15
2.2.3 Workflow and Integration	16
3 Analysis of Algorithms	17
3.1 Random Walk and Agent-Based Generation	17
3.1.1 Theoretical Mechanics and State Space	17
3.1.2 Algorithmic Evaluation	17
3.2 Binary Space Partitioning (BSP) Algorithm	18
3.2.1 Mathematical Space Division and Geometric Partitioning	18
3.2.2 The Connectivity Problem and Delaunay/MST Graph Solutions	18
3.2.3 Algorithmic Evaluation	19
4 Implementation of Generator and UI	20
4.1 System Architecture and Base Class Design	20
4.2 Modular Room Architecture and TileSet Pipeline	21
4.2.1 TileSet Atlases and Aseprite Integration	21
4.2.2 Base Room Structure	22
4.3 Architectural Layout Presets and Directional Biasing	23
4.3.1 Directional Biasing Mechanics	23
4.3.2 Algorithmic Adaptation of Presets	24
4.4 Algorithmic Implementations and Technical Challenges	24
4.4.1 Random Walk Generator and Execution Modes	24

4.4.2	Binary Space Partitioning Logic (BSP.gd)	25
4.5	Room Bitmasking and Dynamic Scene Selection	26
4.6	Room Type Assignment and Reachability Validation	28
4.6.1	Critical Path Identification	28
4.6.2	Distance-Based Spatial Partitioning and Dynamic Scaling	28
4.6.3	Difficulty-Driven Combat and Filler Allocation	29
4.7	User Interface and Interactive Setup	30
4.8	Generated Layout Results	31
5	The Game Prototype	34
5.1	Entity Spawning and Room Instantiation Pipeline	34
5.1.1	Procedural Door Blocker and Collision Management	34
5.1.2	Type-Based Entity Spawning via Marker Nodes	34
5.1.3	Dynamic Performance Optimization: Wake-Up Mechanics	35
5.2	Player Architecture, Combat, and Progression	35
5.2.1	Kinematic Movement and Attack Lunging	35
5.2.2	Knockback Physics and Invulnerability Frames	35
5.3	Enemy AI Archetypes and State Machine Behavior	36
5.3.1	Melee Dash AI (flying_eye.gd)	36
5.3.2	Ranged Artillery AI (FrogEnemy.gd)	36
5.4	Interactive Economic and Upgrade Loop	36
5.4.1	Dynamic Merchant Shop System	36
6	Testing	37
	Conclusion	38

Introduction

Procedural Content Generation (PCG) has evolved from techniques used to mitigate hardware limitations in early video games into a widely adopted approach for generating digital content. As players increasingly seek infinite replayability and diverse experiences, the ability to generate game environments dynamically has become an increasingly important tool for game developers, particularly in projects with limited production resources. By leveraging algorithms to create game worlds, developers can overcome the “content bottleneck”—a common design constraint where the scale and variety of a game world are limited by the time and resources required for manual design [1].

This thesis focuses on the research, design, and implementation of a procedural dungeon generation system within the Godot Engine. The primary objective is to compare the practical suitability of bottom-up and top-down procedural generation approaches for roguelike dungeon design, evaluating whether they can provide sufficiently flexible layouts while maintaining stable performance. While many existing solutions focus on a single algorithmic approach, this work explores and contrasts the comparative strengths of two distinct methodologies representing these paradigms: Random Walk and Binary Space Partitioning (BSP).

The core contribution of this work is the development of a modular dungeon generation framework capable of integrating procedural layout generation with gameplay-oriented systems. By utilizing tile bitmasking logic for door connectivity and integrating navigation systems, this thesis demonstrates how procedural layouts can effectively support gameplay mechanics.

Chapter 1

Theoretical Background of Procedural Content Generation

1.1 Foundations and Taxonomy of Procedural Content Generation

Procedural Content Generation (PCG) represents a fundamental framework shift in modern digital game design. As established in modern game design literature [1, 9, 10], PCG is conceptualized as the algorithmic generation of game content with comparatively limited or indirect input from designers [1]. In the context of modern development, this effectively shifts the designer’s role from directly crafting assets to authoring the generative rules and constraints that govern their creation.

1.1.1 The PCG Taxonomy

According to the core literature [1], procedural generators can be classified across several multi-dimensional axes that dictate their operational constraints and architectural purpose:

- **Online vs. Offline Generation:** This axis specifies the temporal execution of the algorithm. Offline generation occurs during the development cycle, serving as an authoring tool to assist designers. In contrast, online generation occurs dynamically at runtime while the game is being executed by the player [1, 9, 10]. The framework implemented in this thesis strictly utilizes *online generation*, forcing the algorithms to operate within strict runtime computation limits to prevent stuttering or long loading screens during level transitions.
- **Necessary vs. Optional Content:** Content is deemed necessary if it is critical to the progression of the game loop, which means that the player must interact with it to advance [1]. Optional content can be bypassed or ignored. Because the generated dungeons dictate the entire layout, progression paths, and enemy configurations necessary to reach the objective, the system developed in this work handles exclusively *the necessary content*. This places an absolute requirement on the algorithmic robustness to prevent dead-ends.
- **Stochastic vs. Deterministic Realization:** A deterministic generator will always produce the exact same output when given identical initial parameters [1]. A stochastic generator relies on probabilistic factors, producing variations in iterations. This project implements a hybrid approach: the internal calculations are stochastic to ensure unpredictable environments, but they are bounded by a fixed random seed, rendering the entire pipeline reproducible.

- **Constructive vs. Generate-and-Test Methods:** Constructive generation runs exactly once and outputs a final level layout without evaluating its aesthetic or structural properties post-generation [1]. Generate-and-test approaches iteratively produce candidates and pass them through a “fitness function” or validator, discarding failures until a viable result is achieved [1]. The algorithms analyzed in this thesis, Random Walk and Binary Space Partitioning (BSP), are primarily *constructive*, though they are frequently augmented with localized topological constraints to guarantee path connectivity [1, 8, 9].

1.1.2 Dimensions of Control and Parameter Vectors

A critical challenge in PCG is the tension between pure algorithmic randomness and structural controllability. Shaker et al. emphasize the necessity of “parameter vectors” or “random seeds,” which function as dimensions of control over the generator’s search space [1].

Without control vectors, a generator risks producing unreadable structural noise. In this work, controllability is achieved by exposing structural parameters to the developer interface. Variables such as total room volume, minimum aspect ratios for spatial partitioning, and branching limits serve as constraints that bound the generative space, allowing the engine to consistently output structurally sound, fair, and engaging layouts.

1.1.3 The Challenge of Expressive Range

Evaluating a content generator requires analyzing its *expressive range*, the structural diversity of the content it is capable of producing [1, 9]. A common pitfall in constructive algorithms is the tendency to generate “variations on a single theme,” where separate iterations look visually and structurally homogeneous. By comparing an agent-based approach (Random Walk) with a geometric decomposition approach (BSP), this thesis explores how different mathematical foundations expand the expressive range of a dungeon generator, offering both organic, chaotic topographies and highly structured, architectural environments within a single system framework.

1.1.4 Advantages and Challenges of PCG

Although the taxonomy defines the mathematical and systemic constraints of a generator, evaluating PCG requires a clear understanding of its practical advantages and inherent software design challenges, as contextualized by the contemporary game design literature [1].

1.1.4.1 Key Advantages of Algorithmic Generation

Implementing procedural generators offers distinct advantages that address both development overhead and player experience.

- **Infinite Replayability:** As established by Shaker et al., replayability is the most prominent driver for online PCG [1]. By producing unique layout topologies for every session, the player cannot rely on pure spatial memory, which shifts the focus of the game toward active adaptation and mechanical mastery.
- **Data Compression and Resource Efficiency:** Historically popularized by titles like *Elite* (1984), PCG allows vast virtual spaces to be represented by minimal storage footprints [1]. Instead of saving thousands of geometric assets or map files, the software only stores the generative algorithm and seed values, reconstructing the data dynamically on demand.

- **Managing Content Diversity and Scalability:** The demands of modern game development often exceed manual production capabilities. Industries utilize hybrid procedural pipelines (e.g. automated foliage generation with tools like SpeedTree) to populate vast environments with diverse variations, avoiding the high financial cost of manually modeling every environmental asset.
- **Parameter-Driven Customization:** Connecting the generator to the control dimensions allows for adjustment of the difficulty in real-time or aesthetic variations [1]. By feeding specific parameters (such as a difficulty vector or visual theme) into the generator, the system can tailor the experience dynamically to match player performance or specific game states.
- **Foundations for Co-Authoring:** Procedural algorithms frequently serve as “co-designers.” A generator can produce a rough structural skeleton (such as a basic dungeon layout or rough terrain), which human level designers subsequently refine, streamlining repetitive tasks and allowing artists to focus on high-value, hand-crafted details.

1.1.4.2 Inherent Challenges and Limitations

Despite its generative power, PCG is not an absolute solution and introduces substantial design risks and structural failings that must be critically managed:

- **Bounded Generative Space vs. Complex Interaction:** As player interaction mechanics become more sophisticated, designing an algorithm capable of accounting for all mechanical variables grows exponentially complex. In many strategy games, players quickly identify the underlying generation rules and default to repetitive, optimized playstyles, reducing the impact of the randomized environment.
- **Incompatibility with Highly Specialized Mechanics:** Games designed around highly precise physics or spatial puzzles (such as *Portal*) rely on tight, intentional environmental layouts. Generating levels that successfully leverage and reward complex player abilities without breaking the game’s logic remains an open research challenge.
- **The Intellectual Vacuum and Lack of Intent:** A fundamental systemic failing of pure mathematical PCG is that algorithms inherently lack artistic design intent. While a human designer places a structural element to convey meaning, trigger an emotional response, or teach a mechanic, a constructive algorithm merely satisfies numerical constraints. Without careful rule-bounding, this vacuum results in spaces that are mathematically valid but fundamentally soulless and design-deficient.
- **The Dynamic Pacing Bottleneck:** Hand-crafted levels allow for precise, authorial control over the emotional and mechanical pacing of the gameplay loop—alternating between high-intensity combat and periods of structural relief. Procedural generation frequently disrupts this curve, occasionally clustering hazardous encounters too closely together or generating vast, unengaging dead zones that break the game’s narrative and mechanical flow.
- **The Aesthetic Homogeneity and Predictability Problem:** Procedurally generated levels can often feel visually flat, repetitive, or structurally chaotic when compared to hand-crafted design. This phenomenon, sometimes referred to as the “thousand oatmeal bowls” problem, highlights how a generator might produce mathematically unique layouts that nonetheless feel indistinguishable and entirely predictable to the player once the underlying logic is decoded. Achieving the visual and atmospheric cohesion of meticulously designed titles (like *Dark Souls*) is incredibly difficult

for pure algorithms, which is why commercial successes like *XCOM 2* or *Invisible, Inc.* rely heavily on modular, rule-bounded constructive generation to maintain a high artistic standard.

Chapter 2

Technology Stack and State of Art

This chapter outlines the technical environment and tools selected to implement and evaluate the procedural generation algorithms. Since the primary focus of this thesis is the algorithmic design of the spatial generators, the selected development stack is configured to minimize rendering and integration overhead, allowing for rapid iteration and performance profiling.

2.1 Godot Engine

The Godot Engine (v4) was selected as the primary platform for executing and rendering the procedural dungeons. Because the engine is lightweight and open-source, it provides a highly flexible environment for implementing custom algorithms without the bloat of larger commercial engines [4].

2.1.1 The Node and Scene Composition

At the core of Godot’s architecture is the “Scene Tree” hierarchy, where game elements are represented as specialized nodes. Unlike rigid component-based systems, Godot favors composition through nested, self-contained scenes. As evaluated by Bradfield [2], this paradigm allows developers to build complex, modular systems by instantiating independent building blocks.

For a procedural dungeon generator, this composition is highly efficient. Instead of forcing the algorithm to handle complex rendering instructions, each room type is authored as an independent, encapsulated scene file (`.tscn`) containing its own static collision bodies, navigation polygons, and visual assets. The generator operates strictly on a logical grid level – calculating room coordinates and topological connections – and then programmatically instantiates these pre-built scenes into the active Scene Tree [2]. This decoupling ensures that changes to a room’s visual design do not require modifications to the underlying generation algorithms.

2.1.2 GDScript

The logical operations driving the generators are written in GDScript, Godot’s proprietary scripting language. In Godot 4, the introduction of GDScript 2.0 brought critical optimizations necessary for data-heavy algorithmic tasks [3].

While procedural generation inherently relies on intensive mathematical operations and iterative array traversals, older dynamic languages often suffer from performance overhead. As analyzed by Vanhove [3], GDScript 2.0 mitigates this through strict static typing, allowing for compiler-optimized

bytecode execution of spatial and multi-dimensional grid coordinates. This provides the runtime speed necessary to generate complex layouts without causing noticeable load-screen delays.

2.2 Visuals and Pixel Art Aesthetic

The visual presentation of a procedurally generated game is a critical factor in how the player perceives the environment. For this project, a pixel-art aesthetic was chosen not only for its distinct stylistic appeal but also for its technical suitability for modular dungeon generation.

2.2.1 The Role of Pixel Art in Roguelike Design

Pixel art is characterized by its reliance on a limited resolution, where each pixel is deliberate and essential to the visual composition. In the context of procedural generation, this aesthetic offers several advantages:

- **High Modularity:** Pixel art is inherently grid-based, making it perfectly compatible with the tile-based nature of dungeon generators.
- **Artistic Consistency:** By strictly adhering to a defined color palette and grid resolution, even randomly assembled dungeon rooms maintain a cohesive and “hand-crafted” appearance.
- **Readability:** In complex, procedurally generated levels, clear visual communication is vital. The simplified shapes of pixel art allow players to quickly distinguish between interactive elements (like doors or levers) and environmental hazards.

2.2.2 Aseprite

Aseprite was selected as the primary tool for creating the visual assets of this prototype. Aseprite is a specialized editor designed specifically for 2D animation and sprite creation, offering features that optimize the workflow for procedural systems as outlined in its comprehensive documentation [6].

The software utilizes a binary file format (.ase) that strictly structures graphic data into layers, frames, and user-defined tags [7]. Understanding this specific internal file structure is highly beneficial when optimizing workflow integration, as it allows for advanced export operations:

- **Sprite Sheets and Metadata:** Aseprite allows for the efficient creation of sprite sheets (atlases) alongside JSON metadata parsed from frame tags [6, 7]. For a procedural generator, this is essential, as the engine can slice a single PNG file into multiple segments, mapping them to different logical IDs based on data extracted from the visual source.
- **Layered Workflow:** The software supports non-destructive layering, which allows the artist to iterate on environmental details (e.g., adding moss to a wall) without altering the base structure.
- **Pixel-perfect Preview:** Aseprite provides immediate feedback on how assets will appear at specific resolutions, ensuring that when the assets are imported into the Godot Engine, they remain crisp and artifact-free.

2.2.3 Workflow and Integration

The workflow from Aseprite to the Godot Engine is streamlined to support the iterative nature of procedural level design. Assets are exported from Aseprite as optimized PNG sprite sheets and imported into Godot's *TileSet* resource editor.

Within Godot, these assets are processed through a "TileSet Atlas" pipeline, which assigns specific physical collision shapes and navigation data to each tile. Because the generation system relies on modular room scenes, this workflow allows for a rapid update cycle: an artist can modify a sprite in Aseprite, re-export the file, and the Godot generator automatically updates the look of the dungeon without requiring changes to the underlying algorithm or code structure. This efficiency is paramount when fine-tuning the visual variety of the dungeon's environments.

Chapter 3

Analysis of Algorithms

3.1 Random Walk and Agent-Based Generation

The Random Walk algorithm, often formalized in game design as an “agent-based digger” or the “Drunkard’s Walk,” is a bottom-up stochastic process. The algorithm can be modeled as a discrete-time Markov process [12].

3.1.1 Theoretical Mechanics and State Space

In a standard two-dimensional grid environment, the algorithm initializes a generative agent at a starting position $P_0 = (x_0, y_0)$. At each discrete execution step t , the agent transitions to a neighboring state P_{t+1} by selecting a random cardinal direction from the set of available vectors:

$$D = \{(0, 1), (0, -1), (1, 0), (-1, 0)\}$$

The position is updated dynamically via the stochastic function, where d is sampled uniformly from D :

$$P_{t+1} = P_t + d, \quad d \in D$$

As the agent occupies a coordinate, it mutates the cell state from a structural wall to an open, navigable floor tile. The process continues until a termination threshold defined either by a maximum step counter l_{max} or a specific targeted room volume is reached [12].

3.1.2 Algorithmic Evaluation

- **Advantages:** The agent-based approach inherently guarantees local connectivity along the generated path. Because every floor tile is carved sequentially by a generative agent, the resulting layout is free of completely detached, unreachable segments. Under the specific experimental constraints defined by Naußed and Sapokaite, the Drunkard’s Walk algorithm yielded a playable relative space size of 100%, as the sequential carving process prevented the generation of isolated walkable tiles in their testing environment [12]. Additionally, the algorithm is computationally inexpensive; from an asymptotic perspective, its time complexity scales linearly as $O(l_{max})$ where l_{max} is the maximum step threshold, with each iteration requiring $O(1)$ state checks and coordinate mutations under the assumption of constant-time grid access. The worst-case space complexity is similarly bounded by $O(l_{max})$ to store the coordinates of the visited grid cells.

- **Limitations:** While local paths are connected, the global shape is highly unpredictable and difficult to constrain. Without bounding envelopes, the layout can easily clip through pre-defined world boundaries. Furthermore, establishing a structured progression path or logically distributing critical gameplay checkpoints requires separate graph-traversal post-processing, as the basic algorithm does not account for macro-level spatial flow.

3.2 Binary Space Partitioning (BSP) Algorithm

Binary Space Partitioning (BSP) is a top-down geometric subdivision method, often employing randomized split locations in PCG applications to achieve structured, architectural layouts [13].

3.2.1 Mathematical Space Division and Geometric Partitioning

The core mechanic of the BSP algorithm is the recursive subdivision of a continuous or discrete two-dimensional bounding region V_0 into disjoint rectangular sub-regions along randomly selected horizontal or vertical axes [13].

During each recursive pass, the algorithm evaluates the dimensions of a generated sub-region. To keep the generative process stable, the subdivision is governed by defined geometric limits: a maximum division depth, a minimum permissible sub-region width (W_{min}), and a minimum permissible sub-region length (H_{min}) [13]. If a region exceeds these thresholds, a cutting coordinate k is selected within bounded constraints to prevent heavily skewed, narrow slices [13]. The space is then subdivided into two smaller disjoint sub-regions, V_A and V_B , such that:

$$V_A \cap V_B = \emptyset, \quad V_A \cup V_B = V_{parent}$$

The partitioning terminates automatically when no further subdivisions are possible, either because the maximum iteration limit is reached or because the dimensions of all remaining sub-regions fall below the permitted minimum bounds [13]. The finalized sub-regions serve as the spatial boundaries for room placement. However, because these partitioned regions are adjacent and tightly packed, specific geometric constraints, namely room offsets and corner modifiers, are applied to define a structural buffer zone between neighboring walls to prevent room overlaps and reserve space for subsequent corridor integration [13].

3.2.2 The Connectivity Problem and Delaunay/MST Graph Solutions

Because top-down partitioning inherently segments space into geometrically separated room regions, a primary challenge of BSP is corridor generation and connectivity assurance [13]. Naive implementations that attempt to connect adjacent partitions locally often suffer from a lack of reliable routing, which may fail to preserve global connectivity [13].

To resolve this connectivity problem, some modern BSP-based implementations augment the partitioning process with graph-theoretical approaches to establish a reliable topological layout. For analytical purposes, let $G = (V, E)$ denote the complete graph where $V = \{v_1, v_2, \dots, v_n\}$ represents the spatial centers of all valid generated rooms $v_i = (x_i, y_i) \in \mathbb{R}^2$, and E represents the set of all possible straight-line paths connecting these vertices. For each edge $e = (v_i, v_j) \in E$, a scalar weight $w(e)$ is assigned, representing the Euclidean distance between the room centers:

$$w(e) = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$$

From this representation, the topological connectivity is established through a multi-stage graph-theoretic pipeline:

1. **Delaunay Triangulation:** A triangulation $DT(V) \subseteq G$ is computed such that no vertex $v_k \in V$ lies inside the circumcircle of any triangle in $DT(V)$. This step generates a planar graph limiting candidate connections to spatially adjacent rooms, reducing the likelihood of geometrically implausible connections.
2. **Minimum Spanning Tree (MST):** To avoid a chaotic, over-connected maze of corridors, Kruskal's or Prim's algorithm is applied to the triangulation graph $DT(V)$ to extract a Minimum Spanning Tree $T = (V, E_{MST}) \subseteq DT(V)$. The tree T is defined as the acyclic subgraph that spans all vertices while minimizing the total sum of edge weights while preserving connectivity:

$$\text{Cost}(T) = \sum_{e \in E_{MST}} w(e)$$

This guarantees graph connectivity between all room vertices.

3. **Loop Re-insertion:** A pure MST creates a strictly acyclic layout, meaning there is only one valid path between any two points. In game design, this may increase backtracking and reduce navigational flexibility. To optimize exploration and tactical routing, a small subset of discarded non-MST Delaunay edges $E_{cycle} \subset (DT(V) \setminus E_{MST})$ is often deliberately re-inserted into the final graph, depending on the desired topological density of the layout:

$$G_{final} = (V, E_{MST} \cup E_{cycle})$$

3.2.3 Algorithmic Evaluation

- **Advantages:** BSP provides structured spatial distribution and predictable control over environment scale and density [13]. It is well suited for generating structured, architectural layouts and naturally lends itself to room-based progression metrics.
- **Limitations:** The rigid, perpendicular alignment of rooms and corridors can appear blocky or visually repetitive if not masked by diverse art assets. Structurally, BSP does not inherently guarantee connectivity between generated rooms, requiring specialized path-validation frameworks or secondary graph-theoretic routing to resolve [13]. Furthermore, the secondary graph compilation layer increases computational complexity compared to the lightweight Random Walk. From an asymptotic perspective, the geometric partitioning phase operates proportional to the size of the partition tree, which contains $O(2^d)$ nodes in the worst case, where d represents the maximum division depth. This results in both time and space complexity bounded by $O(2^d)$ for the partitioning step. Generating candidate connections via Delaunay triangulation for $n = |V|$ room vertices requires $O(n \log n)$ time. Because the resulting triangulation $DT(V)$ is a planar graph, its edge count is bounded by $|E_{DT}| \leq 3n - 6$, enabling the subsequent Minimum Spanning Tree construction to be completed within $O(n \log n)$ time complexity. Consequently, BSP-based generation trades increased computational complexity for greater control over spatial organization.

Chapter 4

Implementation of Generator and UI

This chapter details the practical implementation of the procedural layout generators, room scenes, and the control interface within the Godot Engine (v4). The system is designed to balance layout controllability with computational performance. Special emphasis is placed on technical challenges encountered during development, including layout entrapment prevention, dynamic tile bitmasking, TileSet integration, and critical path reachability validation.

4.1 System Architecture and Base Class Design

The generation framework relies on an object-oriented GDScript 2.0 structure rooted in an abstract base class, `BaseGenerator`. This class isolates global grid operations, coordinate transforms, and UI parameters from specific spatial generation rules.

Coordinates are mapped dynamically using an origin-centered matrix offset, transforming local space $(-w_{\text{size}}, +w_{\text{size}})$ into positive two-dimensional array indices.

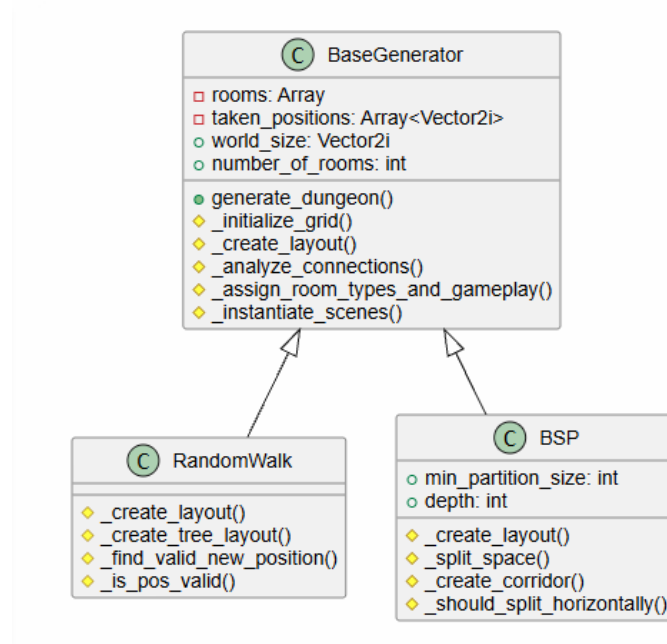


Figure 4.1: UML overview of the generation framework showing the inheritance from BaseGenerator to RandomWalk and BSP.

4.2 Modular Room Architecture and TileSet Pipeline

To bridge abstract grid positions with the visual game environment, generation operates on a two-tier modular asset pipeline consisting of Godot's `TileMapLayer` resource system and a base room scene structure.

4.2.1 TileSet Atlases and Aseprite Integration

Visual assets created in Aseprite are exported as packed PNG sprite sheets and imported into Godot as `TileSet` resources. Using a base grid resolution of 16×16 pixels per individual tile, the generator maps abstract matrix values directly to `TileSet` Atlas IDs.



Figure 4.2: The exported pixel-art TileSet atlas generated in Aseprite, containing wall, floor, and decorative tile variations.

This decoupling ensures that changes to visual textures or tile attributes (e.g., changing wall colors or adding decorative floor tiles) do not require modifications to the underlying procedural generation algorithms.

4.2.2 Base Room Structure

Using the imported TileSet assets, standardized room containers are constructed. Rather than drawing arbitrary geometry at runtime, the dungeon is assembled using these pre-built containers.

To prevent visual overlap and avoid revealing adjacent rooms during gameplay, the generator operates on a fixed spatial footprint of 35×35 grid units per cell. Within this footprint, the actual playable room occupies an approximately 20×20 tile region centered inside the container, while the surrounding space is reserved for doorway corridors and decorative background padding.

The base room container manages the visual representation of walls, floors, and collision layers, serving as the physical boundaries onto which environmental tiles and doorway layouts are assigned based on topological position.

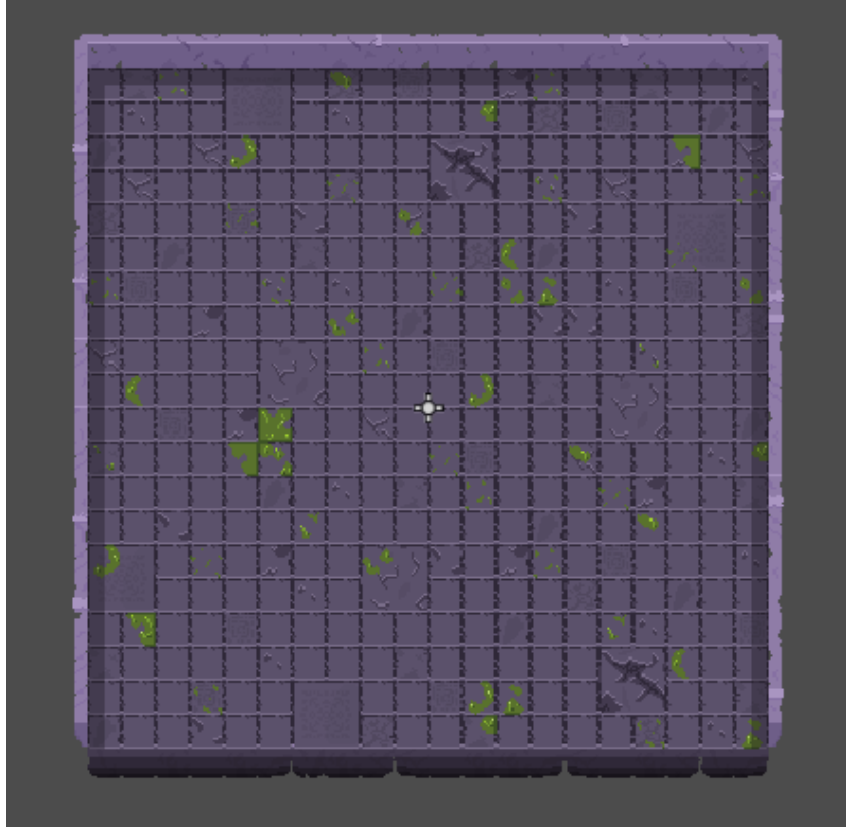


Figure 4.3: Structural layout of a single base room container representing a 35x35 tile footprint.

4.3 Architectural Layout Presets and Directional Biasing

To offer controlled environmental shapes beyond uniform square grids, the generation system incorporates five macro-architectural layout presets: *Basic*, *Tower* (UP), *Cave* (DOWN), *Side* (LEFT/RIGHT), and *Tree*. These presets constrain spatial growth using directional biasing and aspect-ratio modifications.

4.3.1 Directional Biasing Mechanics

Directional bias is governed by the `_get_biased_direction()` method in `BaseGenerator.gd`. Using a uniform pseudo-random roll $r \sim U(0, 1)$, the function maps movement vectors according to weighted probabilities subject to $\sum P(d_i) = 1$:

- **Tower (Preset 1):** Primary direction $\mathbf{d}_{\text{main}} = (0, -1)$ [UP]. Evaluates $P(\text{UP}) = 0.60$, $P(\text{LEFT}) = 0.20$, and $P(\text{RIGHT}) = 0.20$.
- **Cave (Preset 2):** Primary direction $\mathbf{d}_{\text{main}} = (0, 1)$ [DOWN]. Evaluates $P(\text{DOWN}) = 0.60$, $P(\text{LEFT}) = 0.20$, and $P(\text{RIGHT}) = 0.20$.
- **Side Left/Right (Presets 3 & 4):** Primary horizontal vectors $\mathbf{d}_{\text{main}} = (\mp 1, 0)$. Evaluates $P(\mathbf{d}_{\text{main}}) = 0.60$, $P(\text{UP}) = 0.20$, and $P(\text{DOWN}) = 0.20$.
- **Basic / Tree (Presets 0 & 5):** Uniform cardinal sampling with $P(d) = 0.25$ for all cardinal vectors $d \in \{\text{UP}, \text{DOWN}, \text{LEFT}, \text{RIGHT}\}$.

4.3.2 Algorithmic Adaptation of Presets

The generation algorithms process layout presets through distinct structural mechanisms:

4.3.2.1 Random Walk Adaptation

In `RandomWalk.gd`, presets directly manipulate grid bounds and step selection:

1. **Grid Rescaling:** `generate_dungeon()` adjusts `world_size` proportions based on the target room count N . Vertical presets scale grid dimensions to $(N/2, 1.2N)$, forcing narrow, elongated bounding boxes.
2. **Agent Traversal:** In each iteration, the agent samples coordinates via `_get_biased_direction()`, causing the sequential path to dynamically creep toward the preset's dominant axis.

4.3.2.2 BSP Adaptation

Because BSP relies on top-down spatial partitioning rather than sequential steps, `BSP.gd` adapts presets via cutting-plane probabilities and centroid sorting:

1. **Initial Bounding Offset (`start_rect`):** The initial partition bounding box is offset from the origin $(0, 0)$ along the dominant preset vector d_{main} .
2. **Split Orientation Bias (`_should_split_horizontally`):** Vertical presets (*Tower*, *Cave*) evaluate a 75% probability for horizontal cutting planes, stacking sub-partitions vertically into floors. Horizontal presets (*Side*) evaluate a 75% probability for vertical cutting planes.
3. **Centroid Sequence Sorting:** Prior to corridor routing, extracted partition centroids (`room_centers`) are sorted based on Manhattan distance relative to the forced initial exit node, ensuring corridor generation flows sequentially along the preset axis.

4.4 Algorithmic Implementations and Technical Challenges

4.4.1 Random Walk Generator and Execution Modes

The agent-based implementation in `RandomWalk.gd` provides two discrete operational modes:

1. **Standard Stochastic Digger:** Initializes at coordinate $(0, 0)$ and forces an adjacent exit tile to establish an immediate starting path. In each step, the agent selects its origin node using a weighted distribution: 60% probability favors the most recently placed tile (favoring elongated corridor structures), while 40% samples uniformly across all placed tiles (promoting localized room clustering).
2. **Branching Tree Layout (`_create_tree_layout`):** Manages an explicit branch queue (branches). Each branch executes a straight-line traversal for a randomized length $l \in [2, 4]$. Upon branch termination, the endpoint splits into 1 to 3 child branches directed forward, left, or right relative to the parent vector. This recursive splitting generates multi-directional network topologies.

4.4.1.1 The Agent Entrapment Bottleneck

While effective for small maps, generating dense configurations ($N \geq 100$) in standard mode revealed an implementation bottleneck: **agent entrapment**. As open grid space decreases, the probability of an agent sampling an unvisited, valid neighbor drops significantly, leading to excessive resampling attempts and potentially unbounded execution times in pathological cases. To resolve this, a dual-layer safety mechanism was implemented:

1. **Bounded Safety Iterations:** A maximum threshold (`safety_counter < 1000`) prevents engine freezes during generation.
2. **Heuristic Fallback Sampling:** If an agent fails to step forward after 50 attempts, the generator selects a random previously registered room from `taken_positions` to branch anew (`_find_valid_new_posit`

```
# Deadlock resolution loop in RandomWalk.gd
func _find_valid_new_position() -> Vector2i:
    for i in range(50):
        var index = 0
        if randf() < 0.6:
            index = taken_positions.size() - 1
        else:
            index = randi_range(1, taken_positions.size() - 1)

        var base_pos = taken_positions[index]
        var checking_pos = base_pos + _get_biased_direction()

        if _is_pos_valid(checking_pos):
            return checking_pos
    return Vector2i(999, 999) # Return fallback error vector
```

The vector (999,999) represents an invalid sentinel value used exclusively for deadlock detection during execution.

4.4.2 Binary Space Partitioning Logic (BSP.gd)

The BSP implementation combines geometric space division, topological centroid routing, and stochastic room quota completion mechanisms:

1. **Dynamic Depth Allocation:** The maximum recursion depth $d \in [4, 8]$ is dynamically selected according to predefined room-count thresholds to balance partition granularity and generation execution time. Initial bounding coordinates (`Rect2i`) are calculated according to directional layout presets.
2. **Recursive Bisection (`_split_space`):** The spatial region is subdivided into disjoint sub-rectangles. To maintain balanced aspect ratios, axis selection checks if region width exceeds height by a factor of 1.5. Subdivisions terminate when region dimensions fall below `min_partition_size` (2×2).
3. **Centroid Extraction and Corridor Routing:** After the recursive partitioning stage, the geometric center of every leaf partition is extracted. The resulting centroids are stored in the `room_centers` array and sorted according to the selected layout preset. In standard non-tree configurations, the

generator first connects the forced exit room adjacent to the origin with the first centroid in the sorted sequence and subsequently connects each centroid sequentially to the next in the sorted array. The `_create_corridor()` method then traverses the grid one tile at a time along the Manhattan path between both endpoints, registering every intermediate coordinate into `taken_positions`. Consequently, every intermediate corridor coordinate together with the partition centroids contributes to the final dungeon topology.

4. **Quota Fallback Loop:** Strict geometric partitioning occasionally yields fewer leaf nodes than the target room quota N . To ensure that $N = \text{number_of_rooms}$ is satisfied, a fallback loop randomly samples previously registered room positions and attempts to append valid adjacent nodes while respecting world boundaries and start-room constraints until the desired room quota is fulfilled.

4.5 Room Bitmasking and Dynamic Scene Selection

Connecting adjacent rooms manually via conditional branches produces fragile code. To streamline asset placement, a bitmasking algorithm evaluates local grid topology during `_instantiate_scenes()`. Each cardinal neighbor modifies a 4-bit bitmask integer using power-of-two flags:

$$\text{Mask} = (\mathbb{I}_{\text{Top}} \cdot 1) + (\mathbb{I}_{\text{Bot}} \cdot 2) + (\mathbb{I}_{\text{Right}} \cdot 4) + (\mathbb{I}_{\text{Left}} \cdot 8) \quad (4.1)$$

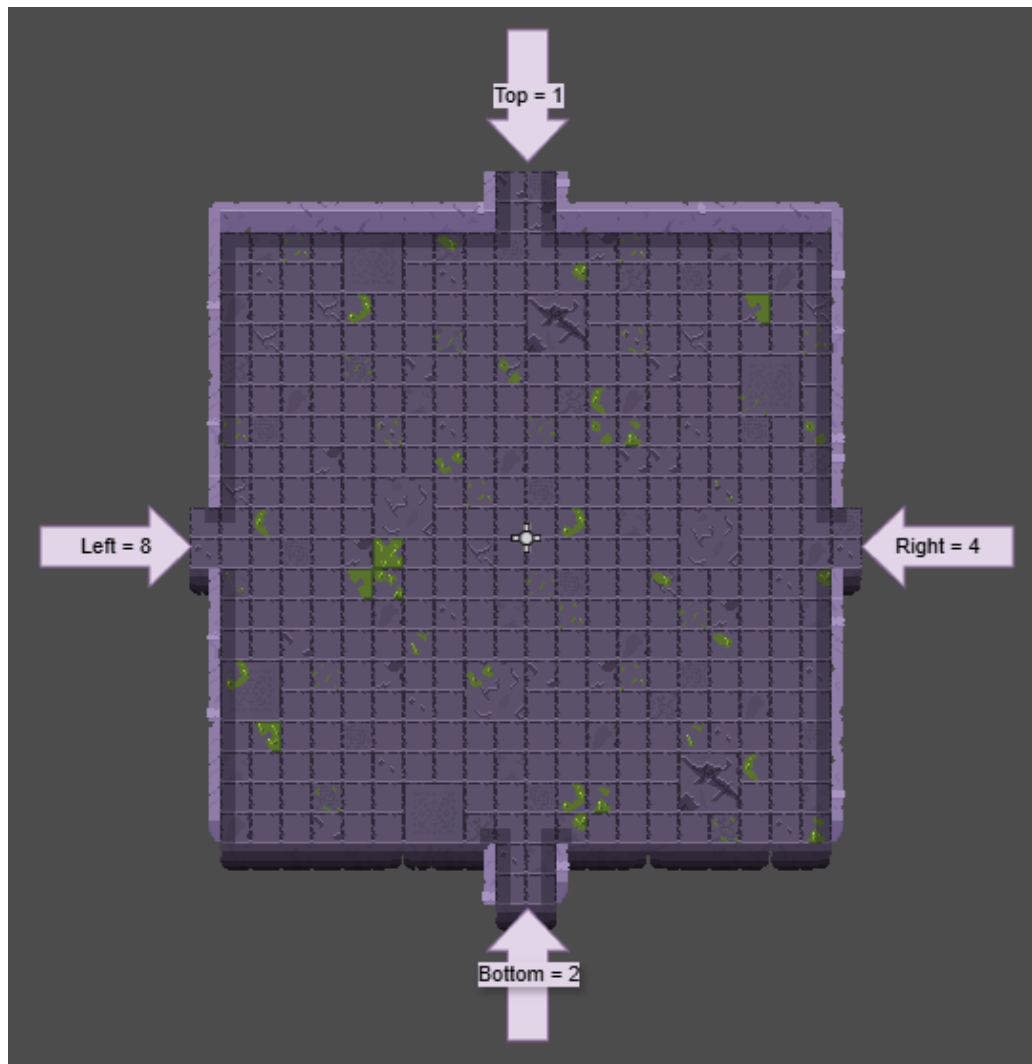


Figure 4.4: Bitmask evaluation diagram mapping directional door combinations to integer keys (1–15) and corresponding pre-authored room scenes.

This calculation yields an integer $m \in [1, 15]$ corresponding to one of 15 non-empty doorway configurations. The key indexes a dictionary (ROOM_SCENES) containing arrays of pre-authored scenes (`.tscn`). A uniform call to `pick_random()` provides aesthetic variation without compromising structural alignment.

```
# Bitmasking calculation in BaseGenerator.gd
var mask = 0
if room_data["door_top"]: mask += 1
if room_data["door_bot"]: mask += 2
if room_data["door_right"]: mask += 4
if room_data["door_left"]: mask += 8

if ROOM_SCENES.has(mask):
    var room_variation = ROOM_SCENES[mask]
    var chosen_scene = room_variation.pick_random()
```

```

var instance = chosen_scene.instantiate()
instance.position = Vector2(pos) * room_pixel_size
map_root.add_child(instance)

```

4.6 Room Type Assignment and Reachability Validation

A major challenge in stochastic PCG is ensuring that critical layout destinations remain accessible while maintaining a balanced gameplay progression. Once the topological layout is fully generated, `_assign_room_types_and_gameplay()` executes a Breadth-First Search (BFS) flood-fill pass starting at origin coordinate (0, 0) to evaluate graph distances and dynamically assign functional room types.

4.6.1 Critical Path Identification

To establish a logical dungeon layout without soft-locking player progression, key topological endpoints are prioritized based on BFS distance and grid connectivity:

1. **Start Room (START):** Fixed at coordinate (0, 0). Serves as the entry point for the player and is assigned a unique visual debug color (Green) for real-time verification.
2. **Boss Room (BOSS):** Identified via `_find_furthest_dead_end()` as the terminal node possessing the maximum BFS graph distance from the Start room. This maximizes the required traversal distance from the entrance.
3. **Key Room (KEY):** Isolated using `_find_key_position()`. The algorithm selects a dead-end node that maintains a minimum path distance ($dist > 3$) from the Start room while maximizing Manhattan distance relative to the Boss room, encouraging additional exploration before reaching the Boss room.

```

# Key placement logic enforcing path isolation in BaseGenerator.gd
func _find_key_position(boss_pos: Vector2i, distances: Dictionary) -> Vector2i:
    var best_pos = Vector2i.ZERO
    var max_dist_from_boss = -1

    for pos in taken_positions:
        if pos == Vector2i.ZERO or pos == boss_pos: continue

        var dist_to_boss = abs(pos.x - boss_pos.x) + abs(pos.y - boss_pos.y)
        var dist_from_start = distances.get(pos, 0)

        if dist_from_start > 3 and dist_to_boss > max_dist_from_boss:
            max_dist_from_boss = dist_to_boss
            best_pos = pos

    return best_pos if best_pos != Vector2i.ZERO else fallback_pos

```

4.6.2 Distance-Based Spatial Partitioning and Dynamic Scaling

All remaining unassigned nodes in `taken_positions` are evaluated based on their BFS distance d_{BFS} from the origin. The remaining positions are sorted in ascending order and split at the median index into two spatial subsets: **Early Spots** (closer to start) and **Late Spots** (deeper in the dungeon).

To ensure that dungeons scale naturally regardless of overall size N , quantities for special utility rooms are computed dynamically as percentage thresholds relative to total room count N and level difficulty D . These scaling coefficients were selected empirically during iterative play-testing to maintain a balanced distribution of utility rooms across varying dungeon sizes:

- **Shop Rooms (SHOP):** $\max(1, \lfloor N \cdot 0.05 \rfloor)$ — allocated primarily to Late Spots to offer late-game resource exchange.
- **Loot Rooms (LOOT):** $\max(1, \lfloor N \cdot (0.15 - 0.02D) \rfloor)$ — prioritized in Early Spots, with availability inversely scaling with difficulty D .
- **Buff Rooms (BUFF):** $\max(1, \lfloor N \cdot 0.08 \rfloor)$ — distributed evenly across both Early and Late partition pools.
- **Trap Rooms (TRAP):** $\max(1, \lfloor N \cdot (0.02 + 0.03D) \rfloor)$ — frequency increases directly with difficulty D .

4.6.3 Difficulty-Driven Combat and Filler Allocation

After placing special utility rooms via `_pop_safe_spot()`, all remaining unassigned nodes are collected into a final candidate pool. The system dynamically balances combat encounters (ENEMY) against passive corridor space (EMPTY) based on the level difficulty parameter $D \in [1, 4]$, where this range was empirically selected to provide increasing combat densities across difficulty levels:

$$P(\text{ENEMY}) = \frac{D}{4.0} \quad (4.2)$$

The total number of enemy rooms is calculated as $N_{\text{enemy}} = \lfloor N_{\text{remaining}} \cdot P(\text{ENEMY}) \rfloor$. The remaining slots are filled with EMPTY designations. This list is shuffled randomly prior to assignment, ensuring that higher difficulty settings produce a denser concentration of combat encounters, whereas lower settings yield more peaceful traversal spaces.

Table 4.1: Abstract room type designations, scaling formulas, and diagnostic debug colors.

Room Type	Dynamic Quota Formula	Debug Color	Topological Role
START	Fixed (0, 0)	Green	Entry node
BOSS	Max BFS Distance	Red	Objective endpoint
KEY	Isolated Dead-End	Yellow	Progression requirement
SHOP	$\max(1, 0.05N)$	Blue	Late-game resource exchange
LOOT	$\max(1, (0.15 - 0.02D)N)$	Gold	Early-game reward
BUFF	$\max(1, 0.08N)$	Cyan	Player enhancement
TRAP	$\max(1, (0.02 + 0.03D)N)$	Dark Red	Disguised environment
ENEMY	Dependent on $P(\text{ENEMY}) = D/4.0$	Purple	Combat challenge
EMPTY	Remaining filler positions	Light Gray	Safe traversal space

An example of this dynamic type distribution applied to a generated layout of $N = 20$ rooms is illustrated in Figure 4.5. The color coding provides instantaneous visual verification of critical path isolation and room type balance.

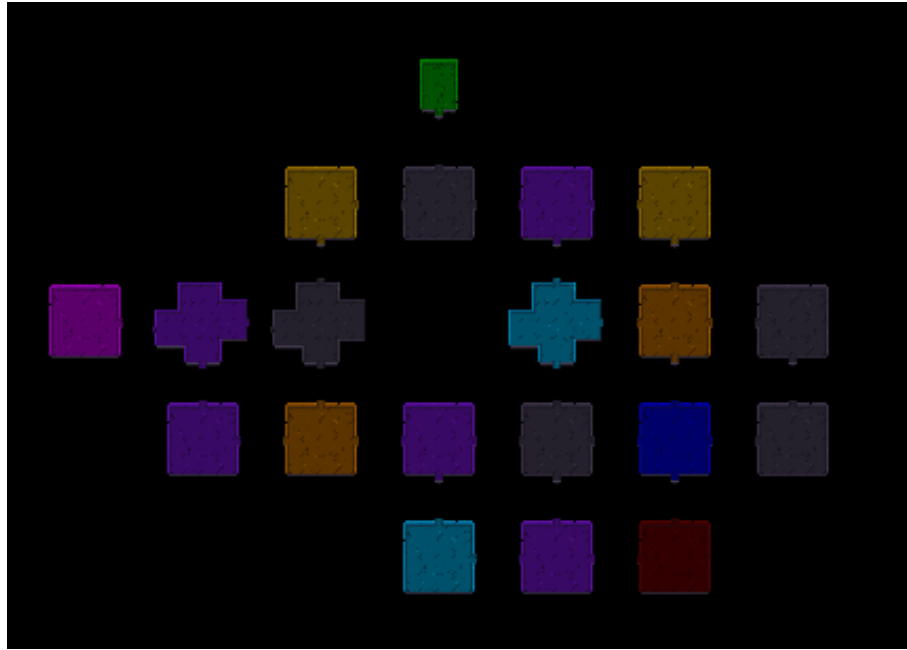


Figure 4.5: A representative generated dungeon layout ($N = 20$) displaying assigned diagnostic debug colors for real-time verification of critical paths and room type distributions.

This abstract categorization ensures that topological generation, distance partitioning, and path validation remain entirely decoupled from concrete entity instantiation. The actual procedural spawning of interactive nodes (e.g., enemy AI instances, loot containers, hazard traps, and shops) within these assigned slots is deferred to the game prototype implementation.

4.7 User Interface and Interactive Setup

The execution environment is driven by a control UI (`Menu.gd`) built using Godot's `Control` node hierarchy.

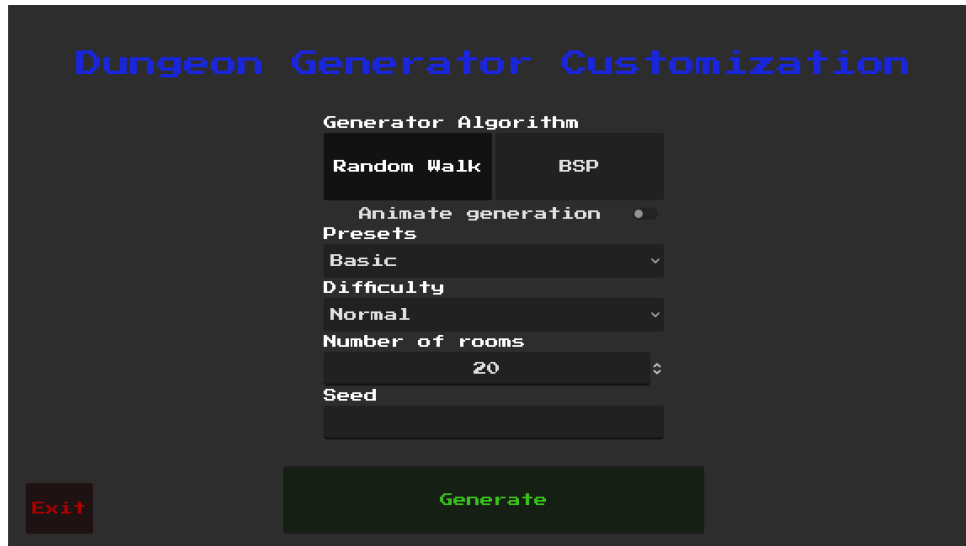


Figure 4.6: The user interface panel allowing real-time parameter tuning, algorithm selection, preset definition, and seed configuration.

The menu passes input parameters to a global singleton (`GeneratorSettings`). Through this control panel, users can dynamically adjust:

- **Generator Algorithm:** Toggle between the `RandomWalk` and `BSP` pipelines.
- **Macro Layout Presets:** Select directional growth behaviors (*Basic*, *Tower*, *Cave*, *Side*, *Tree*).
- **Generation Parameters:** Define target room count N , level difficulty $D \in [1, 4]$, and deterministic random seeds for reproducible layouts.

Separating generation parameters from the underlying generator implementation through the global singleton allows both generation pipelines to operate under a unified configuration interface. This design simplifies parameter propagation while maintaining algorithmic independence between the individual generators.

Upon pressing generate, the selected pipeline initializes visual tilemaps and animated camera transitions, enabling real-time structural inspection and immediate evaluation.

4.8 Generated Layout Results

To demonstrate the practical output of the implementation, complete dungeon structures generated by the algorithmic pipelines were captured using the overview camera in Godot Engine.

Figure 4.7 compares representative outputs produced by the standard `RandomWalk`, the `BSP` partitioning generator, and the branching `Tree` layout preset under identical target room constraints ($N = 20$).

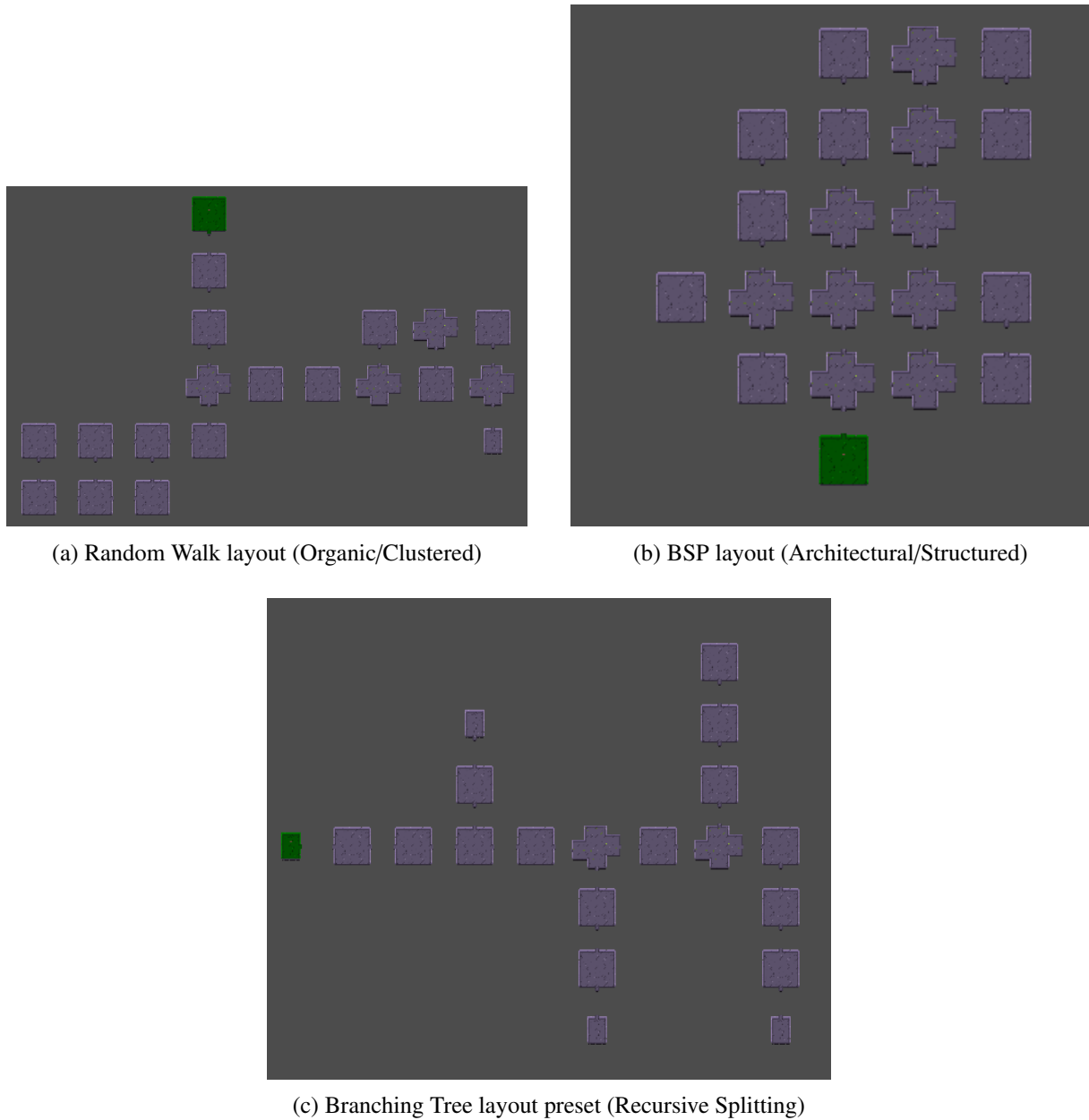


Figure 4.7: Overview comparison of procedurally generated dungeons instantiated with TileSets, bitmapped room variants, and assigned gameplay elements across different operational modes.

The visual comparison highlights the distinct structural characteristics of each generation mode:

- **Standard Random Walk Output:** Displays an organic, winding topology with localized room clusters, loops, and unpredictable boundary contours resulting from stochastic agent decisions.
- **BSP Output:** Produces a structured layout with clear alignment, grid-aligned corridor spines, and a more uniform spatial distribution across partitions.
- **Branching Tree Output:** Demonstrates a distinct multi-directional network topology with clear linear main paths branching into dead-ends and secondary corridors without forming internal loops.

All generated layouts illustrate the integration of tile bitmasking, doorway placement, decorative background generation, and topological reachability validation through the correct assignment of Start, Key, and Boss rooms.

Chapter 5

The Game Prototype

This chapter details the development of the 2D action-roguelike game prototype constructed within the Godot Engine (v4) to demonstrate and validate the procedural dungeon generation pipeline. While Chapter 4 established the topological layout generation, directional presets, and abstract room type assignment, this chapter explores the concrete gameplay systems. It covers runtime entity spawning, player combat mechanics, state-driven enemy AI, upgrade progression loops, dynamic merchant shops, and interactive level navigation.

5.1 Entity Spawning and Room Instantiation Pipeline

The bridge between abstract layout matrices and interactive game environments is managed by the base room controller (`Room.gd`). Upon instantiation by `BaseGenerator.gd`, each room node executes a three-stage initialization pipeline: evaluating doorway topology, spawning room-specific entities, and initializing dormant AI states.

5.1.1 Procedural Door Blocker and Collision Management

Rather than instantiating completely separate wall meshes, pre-authored room templates (`.tscn`) include default entry portals across all four cardinal directions. During the first stage of `setup_room()`, the controller evaluates boolean flags (`door_top`, `door_bot`, `door_left`, `door_right`). If a directional door is absent in the layout topology, the script disables the corresponding `Area2D` portal trigger and enables static collision blockers (`EnemyBlocker`), physically sealing the wall segment.

5.1.2 Type-Based Entity Spawning via Marker Nodes

In the second stage, `Room.gd` evaluates its assigned `RoomType` enum and queries designated marker containers (`Marker2D`) to populate interactive entities. Rather than generating entity positions procedurally at runtime, every room template contains predefined `Marker2D` nodes describing valid, hand-crafted spawn locations:

- **ENEMY:** Queries the `EnemySpawns` node array. The number of spawned enemy instances scales directly with the room's BFS path distance d from origin $(0,0)$. Dungeons deeper than 6 steps spawn 5–7 enemies, whereas early rooms ($d < 2$) contain only 1–2 entities.
- **LOOT:** Instantiates a destroyable `DungeonChest` node containing randomized coin drops, health pickups, and card upgrades at a designated chest marker.

- **KEY:** Spawns the physical key collectable required to unlock the boss encounter door.
- **BUFF:** Spawns a floating card upgrade altar selecting from eight distinct stat-boosting color variants.
- **SHOP:** Instantiates an interactive shop stall node (`shop_stall.tscn`).
- **TRAP:** Spawns a hidden ambush trigger (`trap_trigger.tscn`). Upon player entry, it triggers a wave of 2–3 hostile entities and grants a reward container upon their defeat.

5.1.3 Dynamic Performance Optimization: Wake-Up Mechanics

In the final stage, spawned enemy nodes inherit from `BaseEnemy.gd` and remain dormant by default. To minimize overhead when executing multi-room dungeon layouts, enemy AI processing is restricted exclusively to the player’s currently occupied room.

When the player steps through a doorway portal (`_on_player_transition`), `Room.gd` executes `deactivate_room()` on the origin container, calling `sleep()` to freeze AI processing. Simultaneously, `activate_room()` invokes `wake_up(player)`, passing the active player reference strictly to enemies inside the entered room, significantly reducing unnecessary update calls during gameplay traversal.

5.2 Player Architecture, Combat, and Progression

The player character is implemented as a `CharacterBody2D` node, managing top-down directional movement, melee weapon combat, and resource progression.

5.2.1 Kinematic Movement and Attack Lunging

Player movement utilizes `GDScript`’s `move_and_slide()` combined with `Input.get_axis()`. Cardinal direction vectors are normalized to maintain consistent diagonal speeds. Weapon attacks rotate an offset pivot node (`WeaponPivot`) toward the last facing direction (*up*, *down*, *left*, *right*), enabling a temporary directional lunge effect and activating a sword hitbox (`Area2D`):

$$\mathbf{v}_{\text{lunge}} = \mathbf{d}_{\text{facing}} \cdot \text{lunge_speed} \quad (5.1)$$

When the sword hitbox overlaps an entity implementing `take_damage()`, it applies base damage modified by `GameManager.player_dmg` along with a directional impulse.

5.2.2 Knockback Physics and Invulnerability Frames

Both player and enemy nodes implement directional impulse physics upon receiving damage. When hit, velocity is calculated as:

$$\mathbf{v}_{\text{knockback}} = \text{normalize}(\mathbf{P}_{\text{target}} - \mathbf{P}_{\text{attacker}}) \cdot (\text{strength} + \text{bonus}) \quad (5.2)$$

Deceleration is handled smoothly via `Vector2.ZERO.move_toward()` inside `_physics_process()`. To prevent rapid damage stacking, player hits trigger a temporary invulnerability window augmented by a visual blinking animation.

5.3 Enemy AI Archetypes and State Machine Behavior

Combat challenges are driven by specialized enemy classes deriving from `BaseEnemy.gd`.

5.3.1 Melee Dash AI (`flying_eye.gd`)

Melee units operate a three-stage state loop:

1. **REST:** Decelerates toward zero velocity and waits for a rest timer to expire.
2. **CHARGE:** Modulates sprite color to red, locking onto the player’s position.
3. **DASH:** Launches a high-speed directional impulse (speed = 200) toward the locked coordinates.

5.3.2 Ranged Artillery AI (`FrogEnemy.gd`)

Ranged units feature an independent attack zone (`Area2D`). Upon detecting the player within its attack range, the unit enters a charge state before instantiating a projectile scene (`waterspit.tscn`). Projectiles travel linearly, dealing damage upon player contact or destroying themselves upon colliding with wall tilemaps.

5.4 Interactive Economic and Upgrade Loop

Defeating enemies or breaking chests releases dynamic pickups handled by a global singleton (`GameManager.gd`):

Table 5.1: Card upgrade color mappings and gameplay stat modifiers.

Card Color	Stat Modifier	Mechanic Effect
Purple	+1 Attack Damage	Increases base sword hit damage
Green	+1 Max & Current HP	Expands health container capacity
Cyan	Full Heal	Restores health pool to maximum
Blue	+50 Movement Speed	Accelerates player movement speed
Yellow	+1 Max Coin Drop	Increases loot yield from defeated foes
Red	+5% Heart Drop Chance	Heightens health pickup drop rates
Orange	+30 Knockback Strength	Amplifies damage impulse force
Pink	+0.3s Invincibility	Extends post-hit invulnerability window

5.4.1 Dynamic Merchant Shop System

In rooms designated as `SHOP`, interacting with the merchant stall (`shop_stall.gd`) opens a paused user interface canvas (`shop_ui.tscn`). The shop system generates three random card items with randomized coin costs (15–20 coins), a blind random card roll option, and a reroll feature (5 coins). Purchased upgrades immediately modify `GameManager` parameters while dynamically updating item availability states within the active shop instance.

Chapter 6

Testing

Conclusion

Bibliography

- [1] N. Shaker, J. Togelius, M. J. Nelson: *Procedural Content Generation in Games: A Textbook and an Overview of Current Research*. Springer, 2016.
- [2] C. Bradfield: *Godot 4 Game Development Projects*. Packt Publishing, 2023.
- [3] S. Vanhove: *Learning GDScript by Developing a Game with Godot 4: A fun introduction to programming in GDScript 2.0 and game development using the Godot Engine*. Packt Publishing, 2024.
- [4] Godot Engine Documentation: *Godot Docs*. [Online]. Available: <https://docs.godotengine.org/> [Accessed: July 2026].
- [5] Godot Engine Documentation: *GDScript Basics*. [Online]. Available: https://docs.godotengine.org/en/stable/tutorials/scripting/gdscript/gdscript_basics.html [Accessed: July 2026].
- [6] Igará Studio: *Aseprite Docs*. [Online]. Available: <https://www.aseprite.org/docs/> [Accessed: July 2026].
- [7] Igará Studio: *Aseprite File Format Specifications*. [Online]. Available: <https://github.com/aseprite/aseprite/blob/main/docs/ase-file-specs.md> [Accessed: July 2026].
- [8] M. H. M. Al-Khafaji, et al.: *Procedural 2D Dungeon Generation Using Binary Space Partition Algorithm And L-Systems*. *Journal of Computer Science*, 2024.
- [9] K. Stuart, et al.: *Graph-Based Procedural Dungeon Generation for Roguelike Games*. *International Journal of Computer Games Technology*, 2024.
- [10] S. R. Research, et al.: *Visual Computing and Optimization Frameworks for Procedural Level Generation*. *International Journal of Computing and Multimedia*, Vol. 6(2), September 2025.
- [11] T. Short: *3 Failings of Procedurally Generated Game Design*. Game Developer, 2014. [Online]. Available: <https://www.gamedeveloper.com/design/3-failings-of-procedurally-generated-game-design> [Accessed: July 2026].
- [12] D. Naußed, R. Sapokaite: *Evaluation of Procedural Content Generators for Two-Dimensional Top-Down Dungeon Levels*. Bachelor's thesis, Department of Game Design, Uppsala University, 2021. [Online]. Available: <https://www.diva-portal.org/smash/record.jsf?pid=diva2%3A1569492> [Accessed: July 2026].
- [13] J. M. Yap, N. C. Woo, B. S. Tan, K. S. Chong: *From Algorithm to Playable Space: A Technical Note on BSP-Based Dungeon Design*. *International Journal of Creative Multimedia*, Vol. 6(2), pp. 287–308, 2025.